COMPARISON · SPLIT · STRUCTURE

compare-code

Two fenced code blocks side-by-side, each with a label.

QUERY PATH · REPORT GENERATION

The N+1 query that slowed every report.

BEFORE · ONE QUERY PER ROW

```
const signals = await db.signals.findAll();
for (const s of signals) {
  s.owner = await db.users.find(s.ownerId);
}
return signals;
```

AFTER · ONE BATCHED JOIN

```
const signals = await db.signals.findAll({
  include: { owner: true },
});
return signals;
```

AFTER & BEFORE · COMPONENT MANIFEST LOADING

Folder-shape lookup, with the prior approach for reference.

AFTER · FOLDER SHAPE

```
function loadOne(name) {
  const p = path.join(
    __dirname, 'lib', 'components',
    name, 'manifest.json'
  );
  return JSON.parse(fs.readFileSync(p, 'utf8'));
}
```

BEFORE · FLAT FILE

```
function loadOne(name) {
  const p = path.join(
    __dirname, 'lib', 'components',
    `${name}.json`
  );
  return JSON.parse(fs.readFileSync(p, 'utf8'));
}
```

QUERY PATH · REPORT GENERATION

The N+1 query that slowed every report.

BEFORE · ONE QUERY PER ROW

```
const signals = await db.signals.findAll();
for (const s of signals) {
  s.owner = await db.users.find(s.ownerId);
}
return signals;
```

AFTER · ONE BATCHED JOIN

```
const signals = await db.signals.findAll({
  include: { owner: true },
});
return signals;
```

QUERY PATH · REPORT GENERATION

The N+1 query that slowed every report.

BEFORE · ONE QUERY PER ROW

```
const signals = await db.signals.findAll();
for (const s of signals) {
  s.owner = await db.users.find(s.ownerId);
}
return signals;
```

AFTER · ONE BATCHED JOIN

```
const signals = await db.signals.findAll({
  include: { owner: true },
});
return signals;
```

QUERY PATH · REPORT GENERATION

The N+1 query that slowed every report.

BEFORE · ONE QUERY PER ROW

```
const signals = await db.signals.findAll();
for (const s of signals) {
  s.owner = await db.users.find(s.ownerId);
}
return signals;
```

AFTER · ONE BATCHED JOIN

```
const signals = await db.signals.findAll({
  include: { owner: true },
});
return signals;
```

When NOT to reach for compare-code.

One side is prose. If one column is code and the other is description, use a single fenced block with surrounding prose. compare-code is for code-versus-code.

Snippets longer than 14 lines. The text shrinks below readability past 14 lines per side. Split into two slides or extract the key delta into a smaller diff.

Three-way comparison. compare-code is binary. For three configurations or three implementations, use prose with successive fenced blocks or a `compare-table`.

RELATED COMPONENTS

See also.

`compare-prose` — the change is state, not code

`compare-prose` — the comparison is prose-versus-prose

`redline` — the change is in verbatim text or legal language

`compare-table` — three or more variants on shared dimensions